Assignment - 2

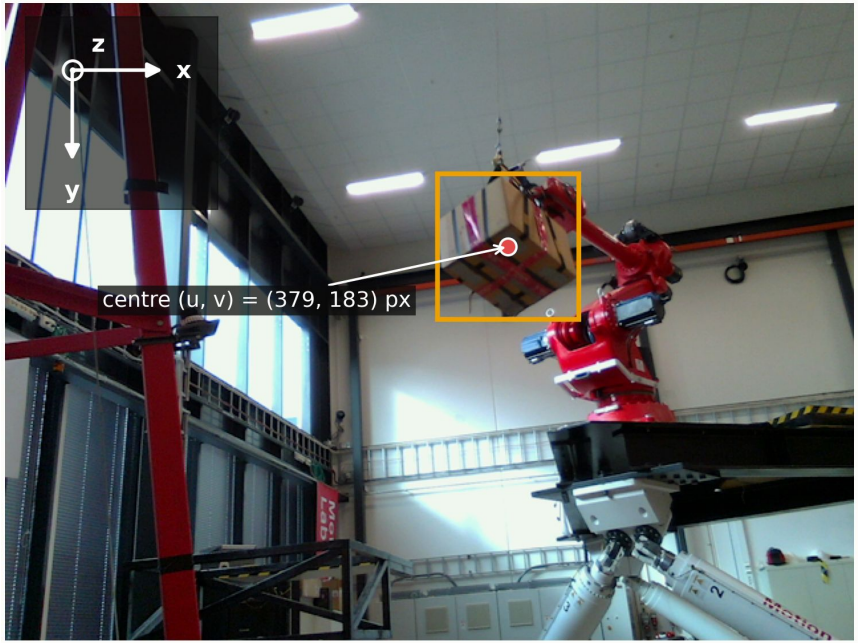
Group - 1

UiA Universitetet i Agder

Sammen om kunnskap

Q1 - Objective

- Find the 3D centre of the swinging load in every frame using a L515 Realsense camera.
- Compare two intrinsic matrices, one from the lab M_int and the M_d from the assignment.
- Decide which matrix is accurate, and why.
- Train an ML model and measure how well it maps one result onto the other.
- Measure the error between the trajectories.



Q1 - Overview

repeated for every frame (270 frames with a valid depth)

1 Detect

YOLO from Assignment 1 finds the load → centre (u, v)

2 Depth

depth d at (u, v): median of 11×11 px, skip pixels with no reading

3 Back-project

$p_c = \tilde{M}^{-1}[du, dv, d, 1]^T$

$M_{int} \rightarrow o_1$ (a)

$M_d \rightarrow o_2$ (b)

4 Learn f_θ

$f_\theta(o_1) = o_2^*$

one-layer perceptron

Adam, 80/10/10 split (train/val/test)

5 Compare

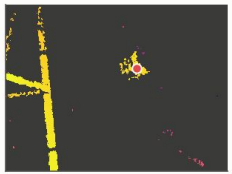
(c) field of view vs L515 datasheet

(e) $\|o_1 - o_2\|$

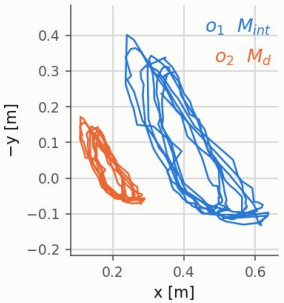
$\|o_1 - o_2^*\|$



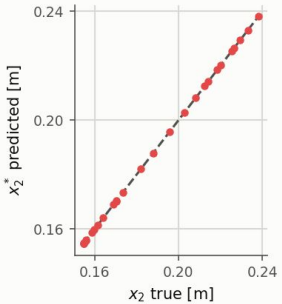
frame 30: box = load, dot = centre (u, v)



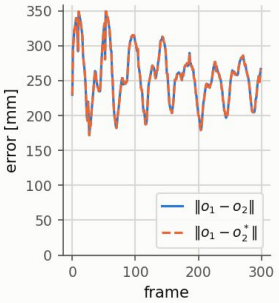
bright = near, grey = no reading (4 % of pixels valid)



front view of the swing: o_2 ~2× smaller than o_1

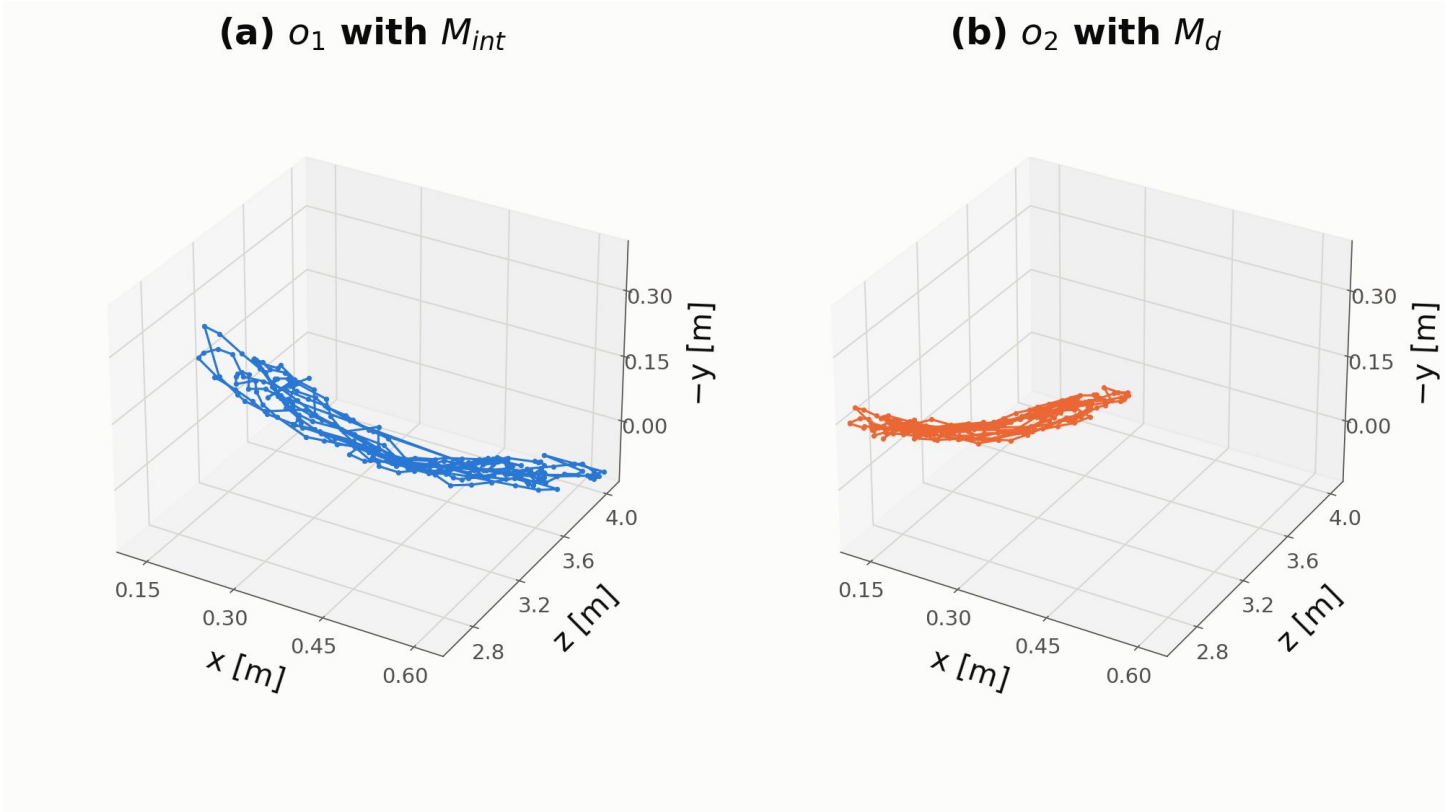


test set on the diagonal: R^2 = 1.0000



FOV M_{int} 56°, M_d 27° (sheet 55°) mean error 255 mm

Q1 - Tracking Results of each intrinsics.



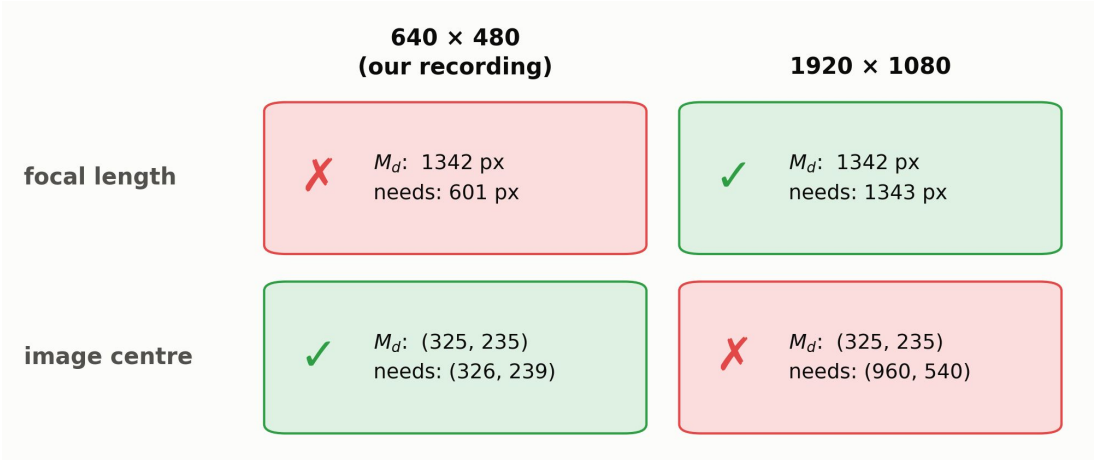
Q1 - Which intrinsics are more accurate?

- M_int matches the L515 camera datasheet, while M_d is around 2 times too narrow.
- M_int was calibrated for this exact camera using the python library from Intel, Pyrealsense2.
- M_d seems to be for a 1920x1080 video, while our video is 640x480.
- O_2 is the wrong one, it places the load closer to the optical axis than it is.

	f_x, f_y [px]	c_x, c_y [px]	field of view H × V	load width	matches reality?
M_{int} **from our L515**	601, 602	326, 239	56° × 44°	0.48 m	✓ **yes**
M_d **default matrix**	1342, 1342	325, 235	27° × 20°	0.21 m	✗ **no ~2× too small**
reality			~55° × 43° (datasheet)	~0.5 m (lidar, Q2)	**reference**

Q1 - Why is M_int more accurate?

- **M_d mixes two resolutions:** its focal length and image centre don't belong to the same image size.
- **Focal length (1342 px):** matches a 1080p stream, but is ~2× too large for our 640×480 recording.
- **Image centre (325, 235):** matches our 640×480 recording, but would be wrong for 1080p (≈ 960, 540).
- **Consequence:** with Z taken from depth, X = (u − c_x)·Z / f_x, so M_d shrinks X and Y by ~½ while Z is unchanged, and o_2 becomes a scaled-down copy of o_1.
- **Conclusion:** M_int was calibrated on our actual setup and resolution, so it's the accurate one; M_d isn't valid for either resolution without correction.



Q1 - Linear Model

1 Data

one pair (o_1, o_2)
per frame, 270 pairs
input o_1, target o_2

2 Split

in time order
train 216 frames
validation 27
test 27

3 Train

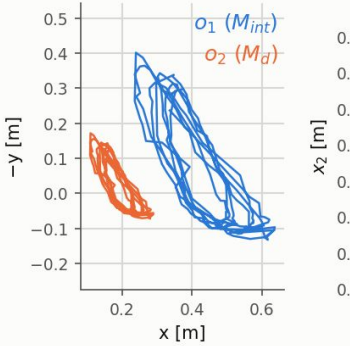
one-layer perceptron,
no activation:
$o_2^* = Wo_1 + b$
Adam, 500 epochs

4 Learned W

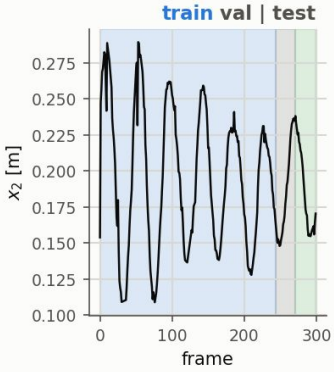
the weights after
training, read back
as a 3×3 matrix

5 Test

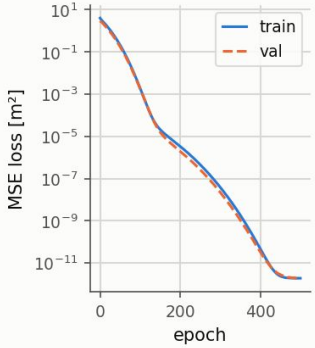
$o_2^* = f_\theta(o_1)$ on the
27 test frames,
compared with o_2



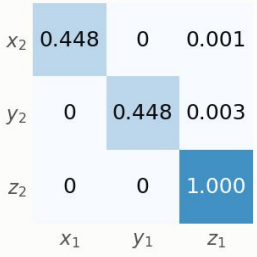
front view of both trajectories



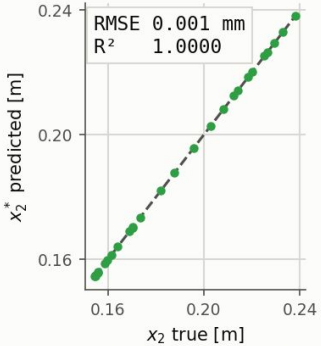
80 / 10 / 10 in time order,
not random: no near-copies



train and val fall together
and flatten: no overfitting



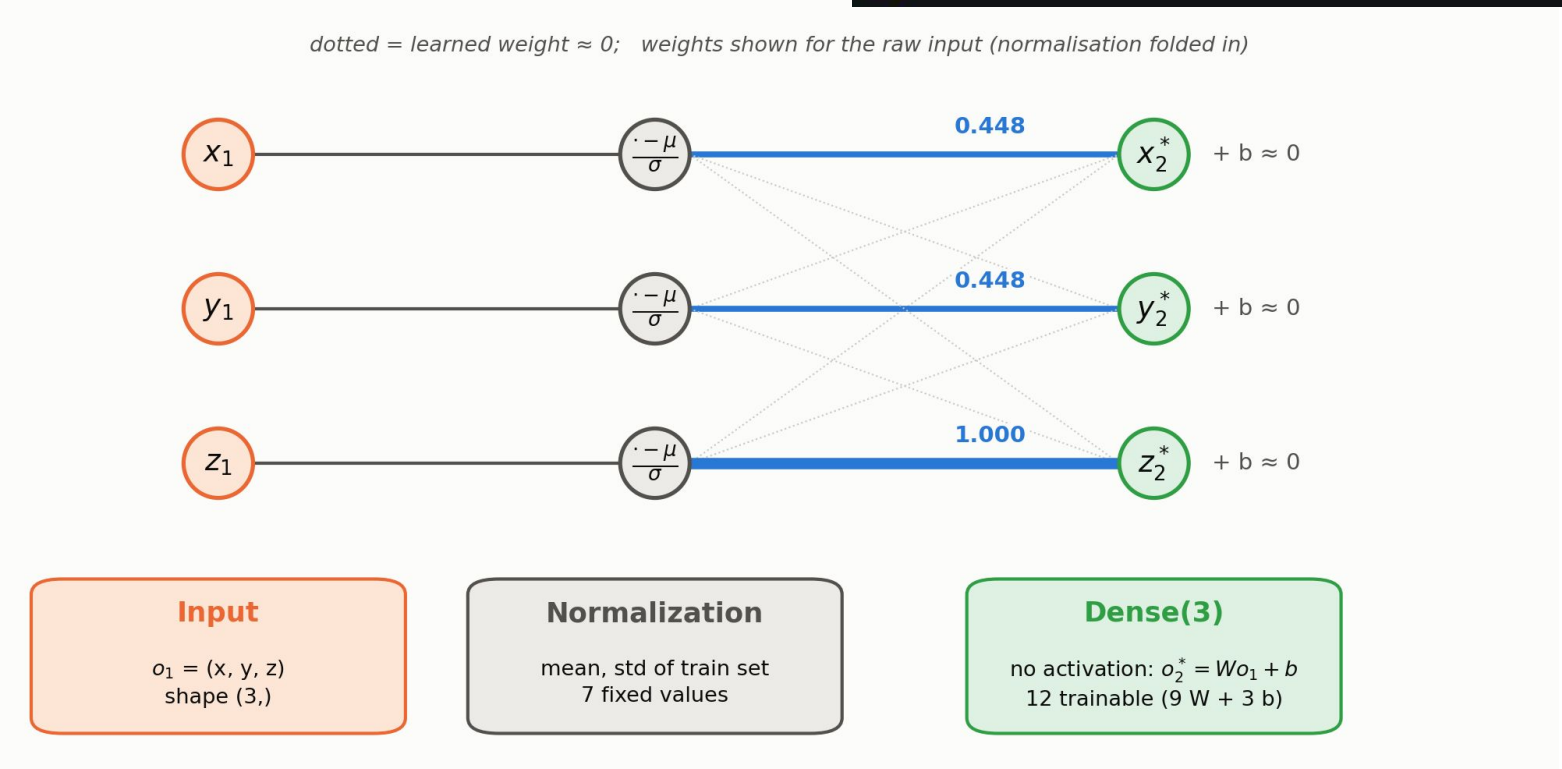
0.448 = f_{int}/f_d on x, y
$z_2 = z_1$, the rest ≈ 0



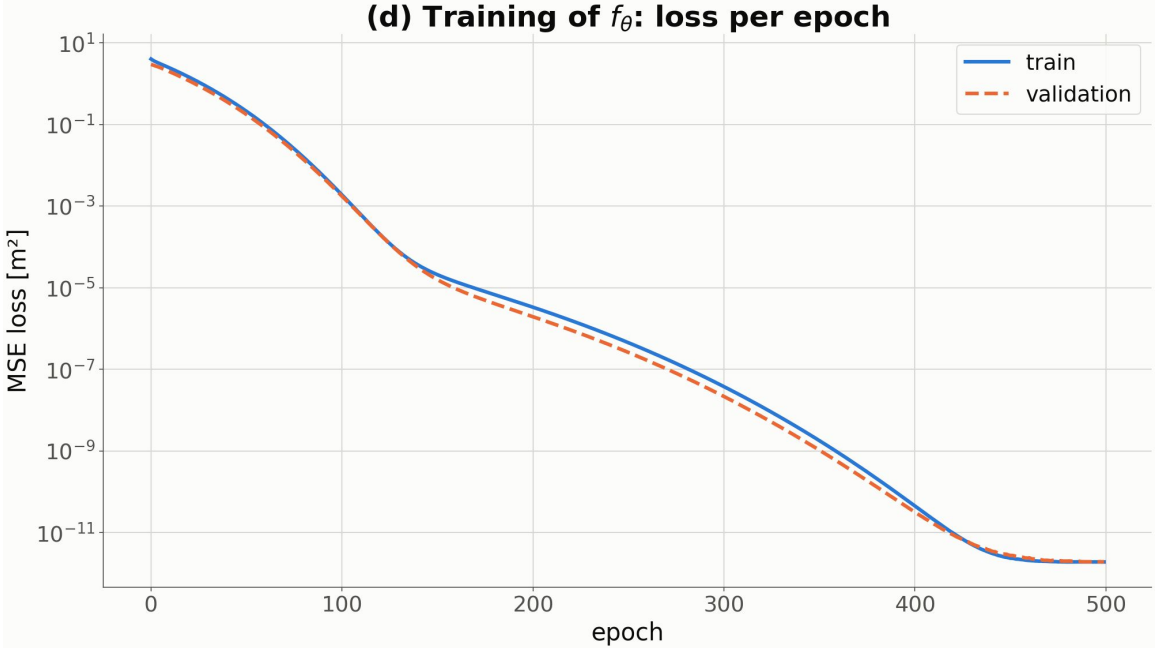
every test frame on the diagonal:
o_2 is a linear function of o_1

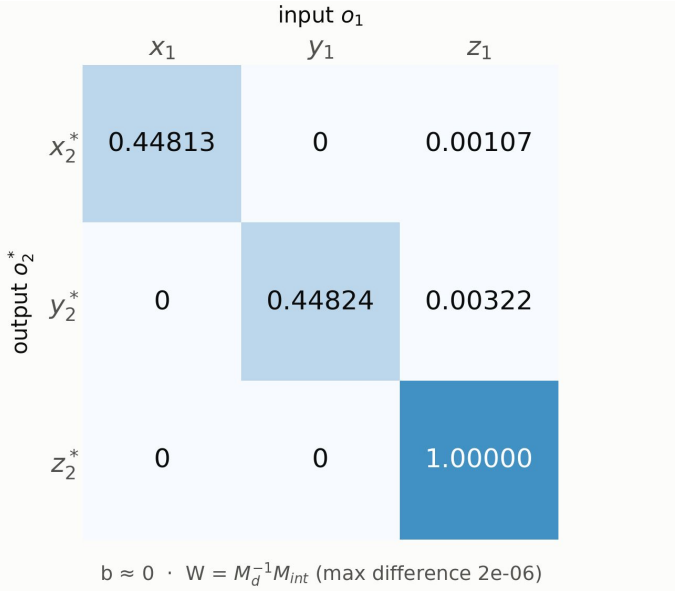
Q1 - Model Architecture

```
self.model = tf.keras.Sequential([
    tf.keras.Input(shape=(3,)),
    normalization,
    tf.keras.layers.Dense(3),
])
```



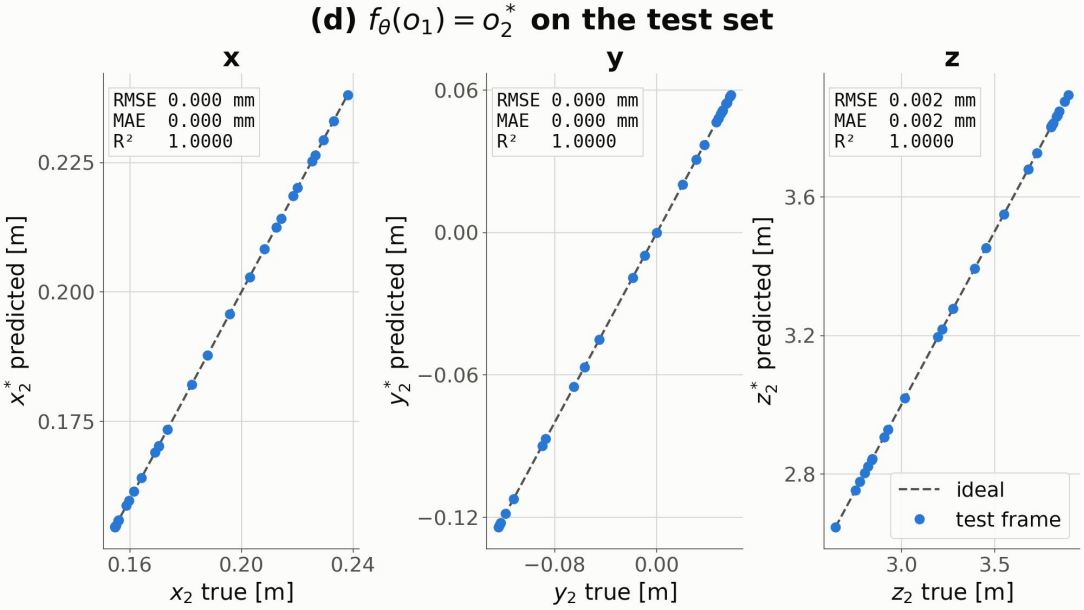
Q1 - Model Training Results





Q1 - Linear Model Results

- **RMSE ≈ 0mm, MAE ≈ 0.002mm, R^2 = 1.000**
- **Why it's perfect:** o_1 and o_2 come from the same pixel (u, v) and the same depth Z; only the intrinsics differ.
- **o_2 is an exact linear function of o_1:** $X_2 = (f_1/f_2)\cdot X_1 + ((c_x1 - c_x2)/f_2)\cdot Z_1$ (same form for Y), and $Z_2 = Z_1$.
- **So the model isn't learning geometry,** it's just recovering the fixed ratio between M_int and M_d.
- **Caveat:** this only holds because both datasets share the same measurements.



Q1 - Conclusion

Results

- Same pixels and depth, but o2 comes out about half the size of o1.
- M_int is correct: its field of view and load size match reality. M_d don't.
- The model learned the conversion between the two matrices almost exactly.
- Using M_d puts the load about 25 cm off.

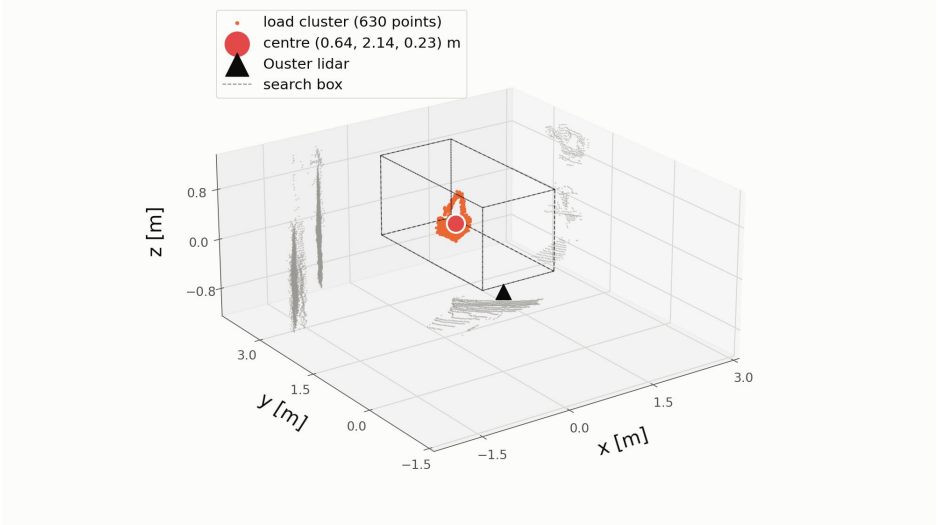
Takeaways

1. Use the camera matrix that matches your resolution.
2. A wrong matrix can still give a believable result.
3. A model can't fix a wrong target. It learns the error too.

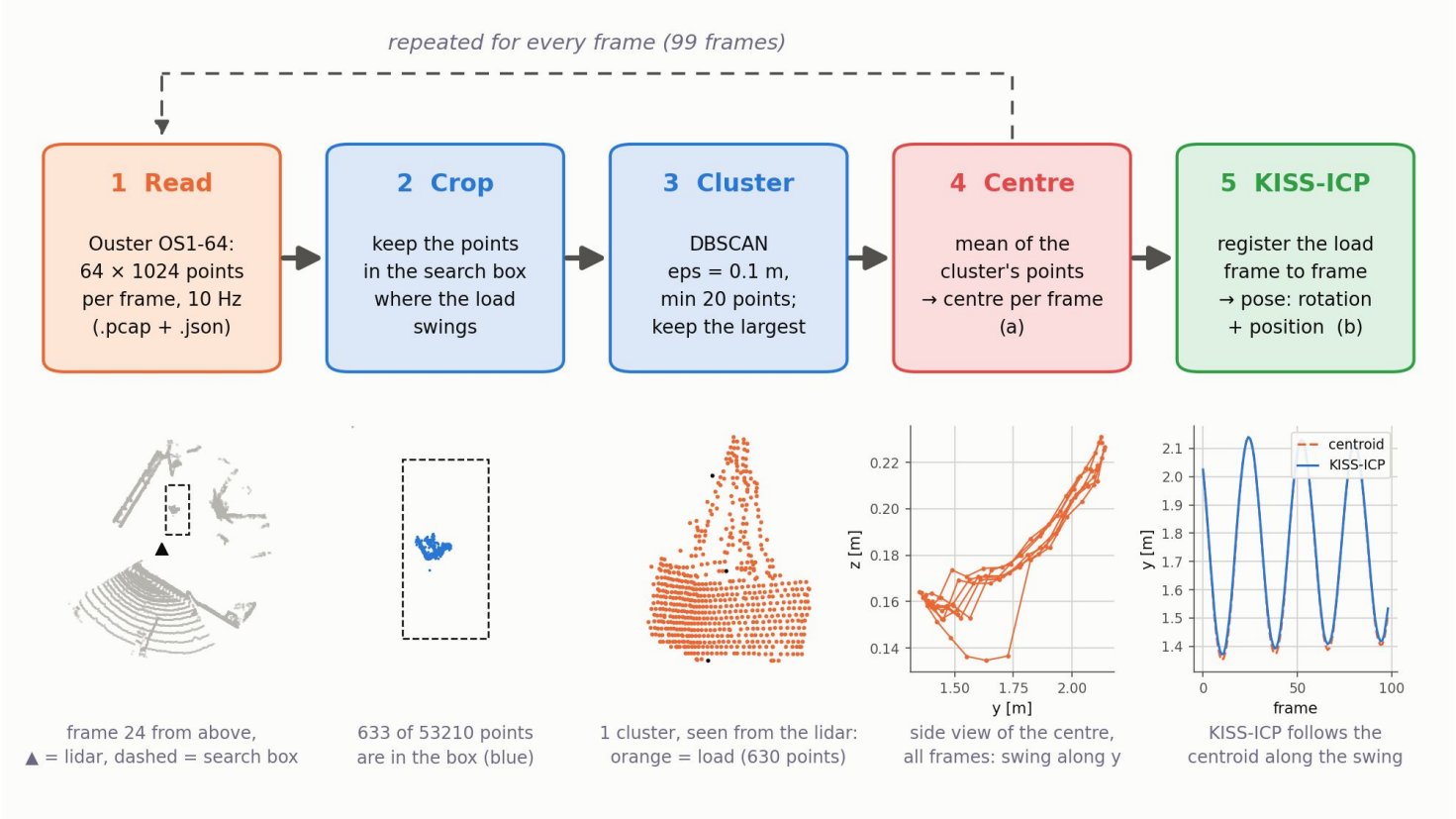
	f_x, f_y [px]	c_x, c_y [px]	field of view H × V	load width	matches reality?
M_{int} **from our L515**	601, 602	326, 239	56° × 44°	0.48 m	✓ **yes**
M_d **default matrix**	1342, 1342	325, 235	27° × 20°	0.21 m	✗ **no ~2× too small**
reality			~55° × 43° (datasheet)	~0.5 m (lidar, Q2)	**reference**

Q2 - Objective

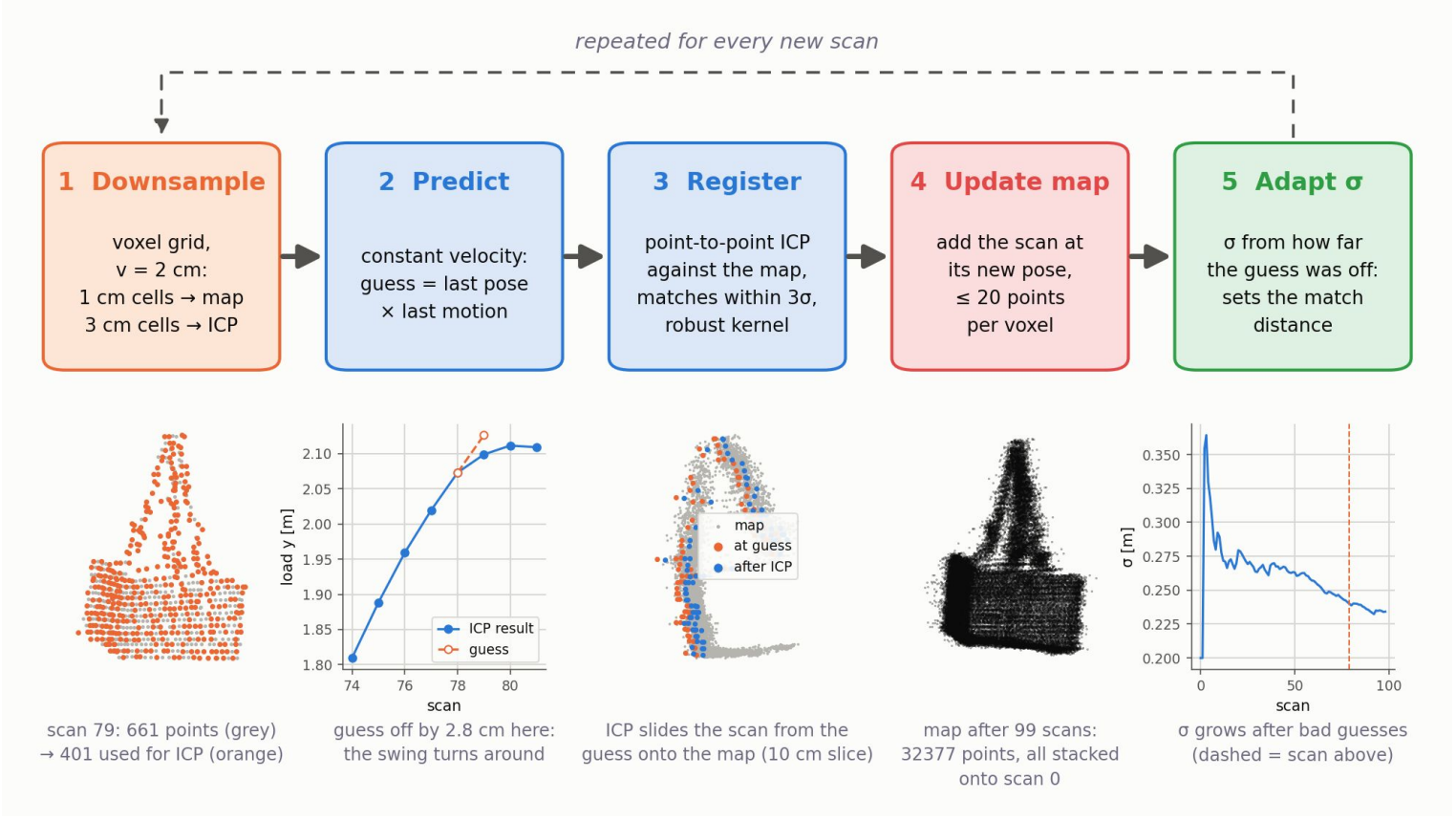
- Find the 3D centre of the swinging load in every frame using an Ouster lidar.
- Separate the load from the rest of the scan by clustering the point cloud.
- Plot the centre of the load for all frames.
- Track the pose of the load between consecutive frames using KISS-ICP.
- Explain how KISS-ICP works, what it is used for, and its drawbacks.



Q2 - Overview


repeated for every frame (99 frames)
1 Read
Ouster OS1-64:
64 × 1024 points
per frame, 10 Hz
(.pcap + .json)
2 Crop
keep the points
in the search box
where the load
swings
3 Cluster
DBSCAN
eps = 0.1 m,
min 20 points;
keep the largest
4 Centre
mean of the
cluster's points
→ centre per frame
(a)
5 KISS-ICP
register the load
frame to frame
→ pose: rotation
+ position (b)
z [m]
y [m]
centroid
KISS-ICP
frame
frame 24 from above,
▲ = lidar, dashed = search box
633 of 53210 points
are in the box (blue)
1 cluster, seen from the lidar:
orange = load (630 points)
side view of the centre,
all frames: swing along y
KISS-ICP follows the
centroid along the swing

Q2 - How does KISS-ICP work?



scan 79: 661 points (grey)
→ 401 used for ICP (orange)

guess off by 2.8 cm here:
the swing turns around

ICP slides the scan from the
guess onto the map (10 cm slice)

map after 99 scans:
32377 points, all stacked
onto scan 0

σ grows after bad guesses
(dashed = scan above)

Q2 - KISS-ICP uses and drawbacks

Uses

- Lidar odometry: estimating how a robot, car or drone moves from its lidar scans.
- The first stage of mapping and SLAM (building a map while tracking the sensor's position in it).
- Works with almost any lidar and needs no camera or IMU.
- Few parameters to tune, and it runs in real time.

Drawbacks

- Drifts over time: there's no loop closure (recognising a place it has already visited), so small errors add up.
- Assumes the scene stands still: moving objects confuse it.
- The constant-velocity guess fails when the motion changes suddenly.
- Needs shapes to lock onto: long corridors, open fields and flat walls give poor results.
- Only gives the pose relative to the first scan, not a position in the world.

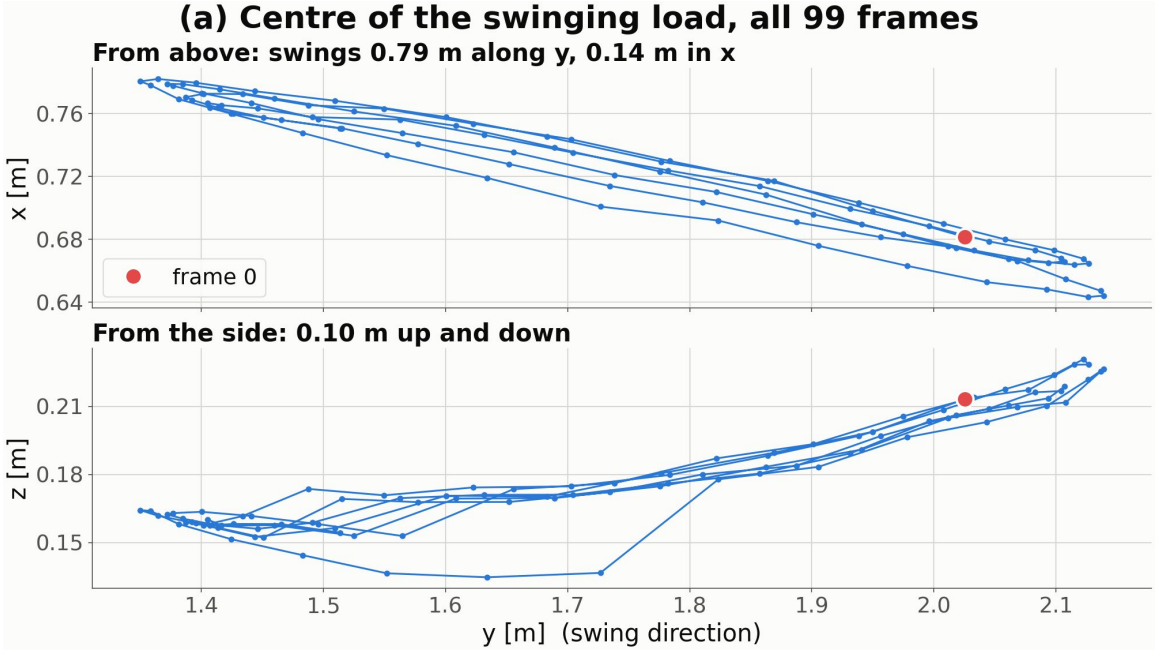
In the assignment

- We used it the other way round: the lidar stood still and the load moved.
- That only works because we gave it the load's points alone (after crop and DBSCAN).
- We saw one drawback directly: the guess was off by up to 2.8 cm at the turns of the swing.

Q2 - Result, Centre of Payload

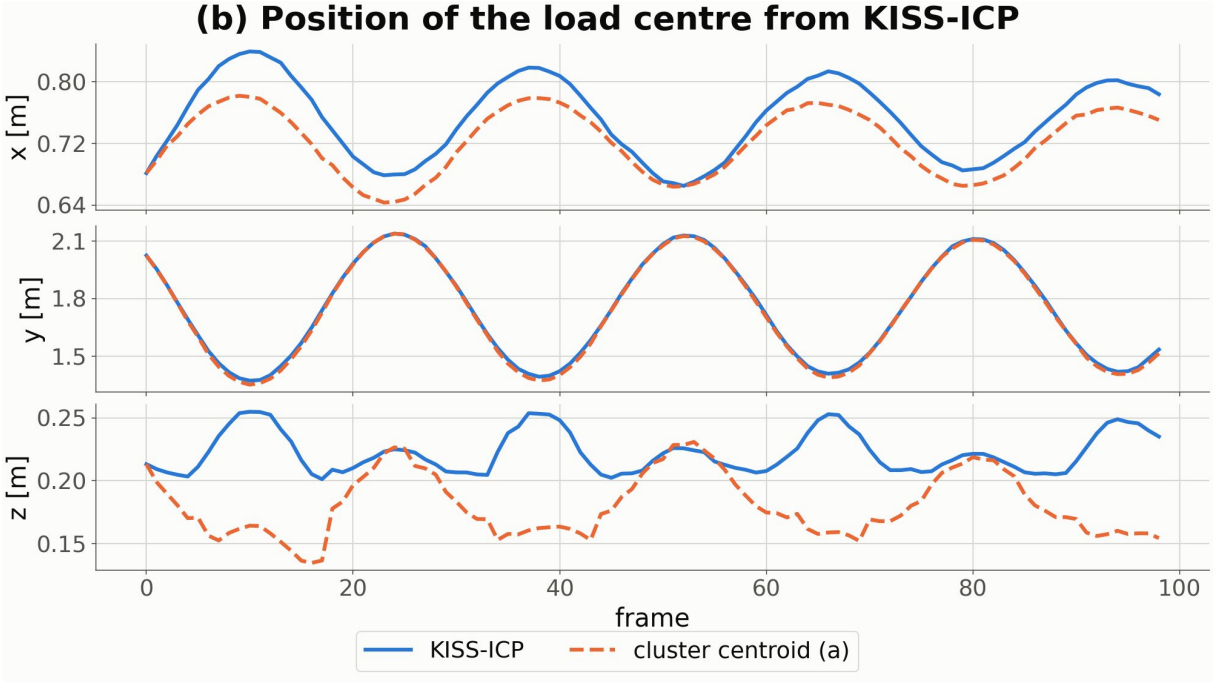
- The load was found in all 99 frames, with 630–1030 points per frame.

- It swings 0.79 m along y (1.35 → 2.14 m), and moves only 0.14 m in x and 0.10 m in z.



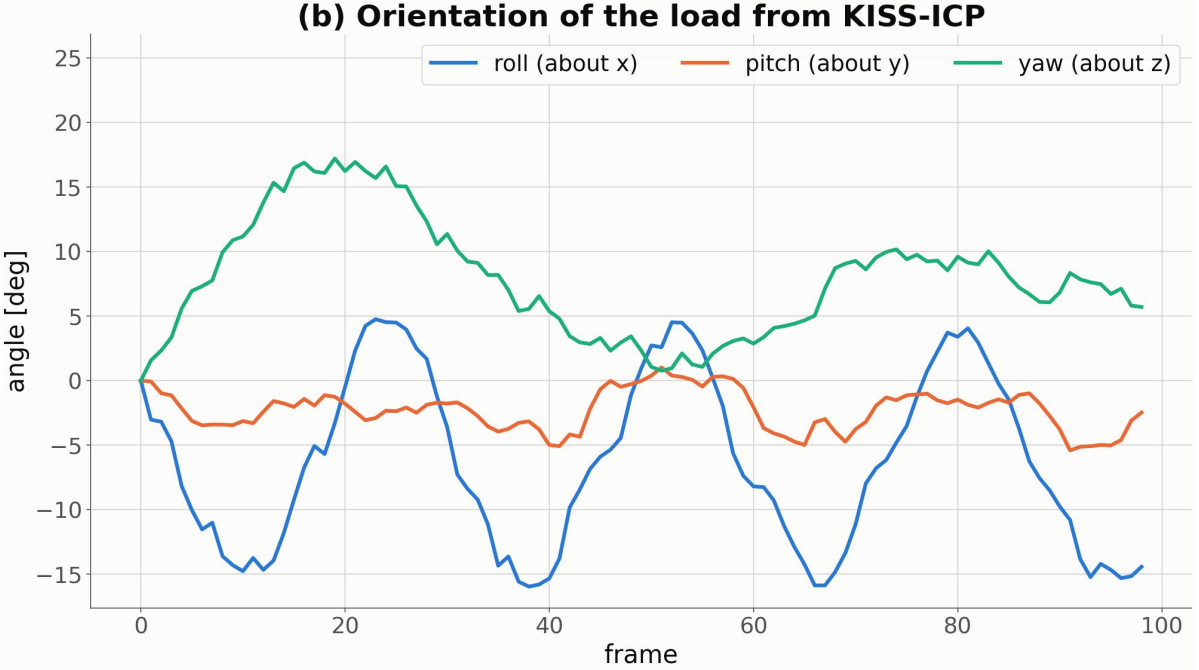
Q2 - Results, Positions from KISS-ICP

- In y (the swing) they agree closely: median 0.8 cm apart, at most 2.2 cm.
- In x and z they differ by about 3 cm (median), up to 6.5 and 9.4 cm.
- The lidar only sees the side of the box facing it. The centroid is the average of the visible points, so it shifts when the box turns. KISS-ICP tracks the whole box as one rigid object.



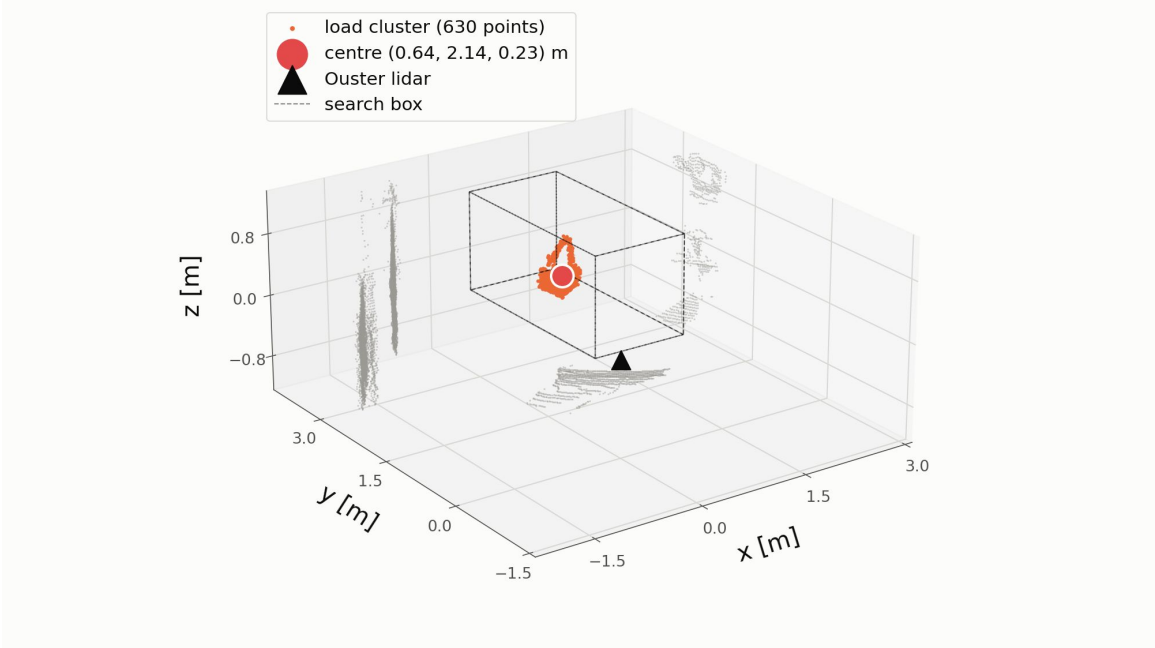
Q2 - Results, Orientation from KISS-ICP

- Roll (about x): from −16° to +5°, in step with the swing.
- Yaw (about z): up to 17°, so the load also twists.
- Pitch is small, −5° to +1°.
- The centroid can't show any of this. Orientation is what KISS-ICP adds over using only the center of the load.



Q2 - Conclusion

- The load swings 0.8 m along y, every 2.8 s.
- KISS-ICP tracks the position to within a few cm of the centroid.
- KISS-ICP also gives the rotation, which the centroid can't.
- The centroid is biased by which side the lidar sees; KISS-ICP isn't.



Q3 - Camera–LiDAR fusion on KITTI

- (a) Explain the KITTI dataset: cameras, LiDAR, calibration.
- (b) (i) Project the LiDAR points onto the image, with depth as color.
- (b) (ii) Detect objects, draw bounding boxes and show their distance.
- (b) (iii) Compare the distances with the ground truth.

Task a:
KITTI dataset: Getting familiar

- Car that drives around with sensors
  - Stereo grayscale camera
  - Stereo color camera
  - Velodyne HDL-64E LiDAR
  - GPS/IMU (OXTS RT3003)

- Dataset used for benchmark of:
  - 2D and 3D object detection
  - Stereo and depth estimation
  - Optical flow
  - Visual odometry and SLAM
  - Object tracking

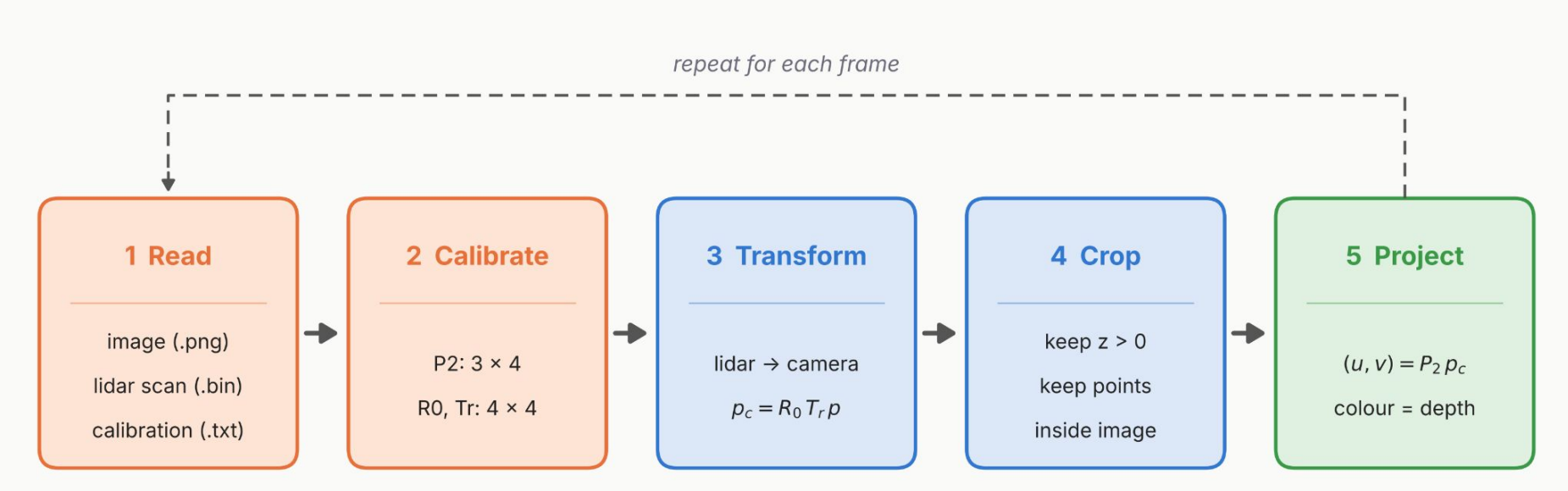


Task b (i):
Sensor fusion between lidar and rgb camera

- Data set:
  - image_2/ – color image from camera 2 (1242 × 375 px)
  - velodyne/ – LiDAR point cloud (x, y, z, reflectance)
  - calib/ – calibration matrices for that frame
  - All data is labeled with same name of all files eg: 00001.txt/.png
- Each sensor has its own frame:
  - Lidar: (x= forward, y= left, z=up)
  - Camera: (x= right, y= down, z= forward (into image))
  - Use Tr_velo_to_cam to transform lidar coordinates to camera

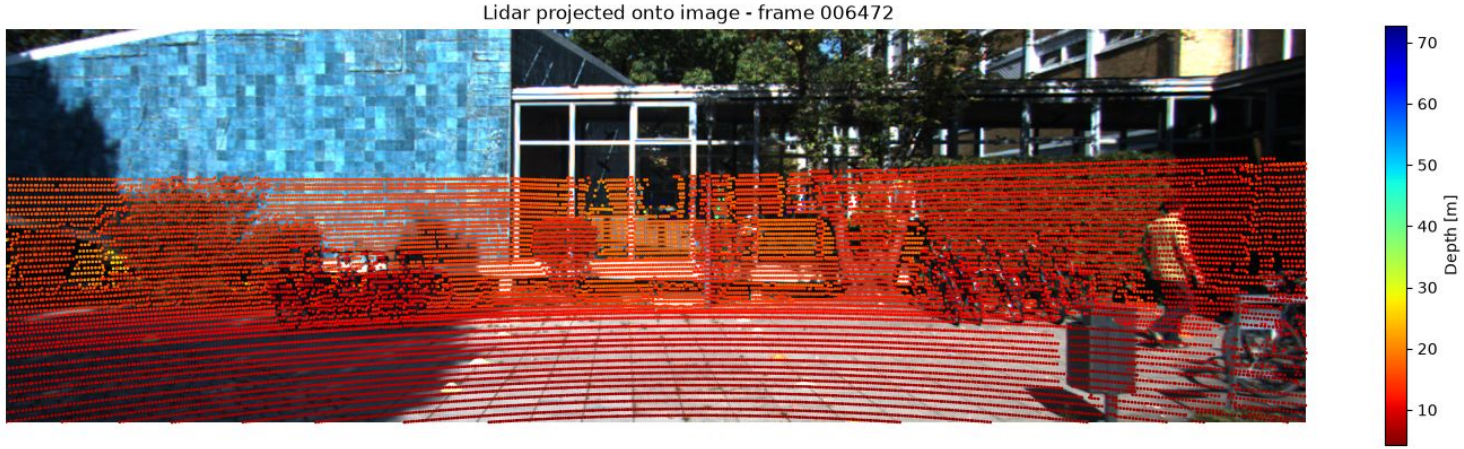
P0, P1	Grayscale camera
P2	Projection into camera 2 (intrinsic)
P3	Projection, right colour camera
R0_rect	Rectification rotation
Tr_velo_to_cam	LiDAR to camera (extrinsic)
Tr_imu_to_velo	GPS/IMU to LiDAR

Task b (i):
Sensor fusion between lidar and rgb camera



Task b (i):
Sensor fusion between lidar and rgb camera

- Only need the points inside the image since lidar is 360 deg and camera is 90 deg
- Then project the lidar frame onto the camera frame (image) while displaying depth
- 5 random images:



Task b (ii):
Detection and segmented display

Goal: Detect the object in the scene, place bounding box around it, measure the distance of object and display the lidar points in the bounding box

Model used: COCO trained yolov11 nano

Classes detected: 0 (People) and 2 (cars)

YOLO11 Model Sizes

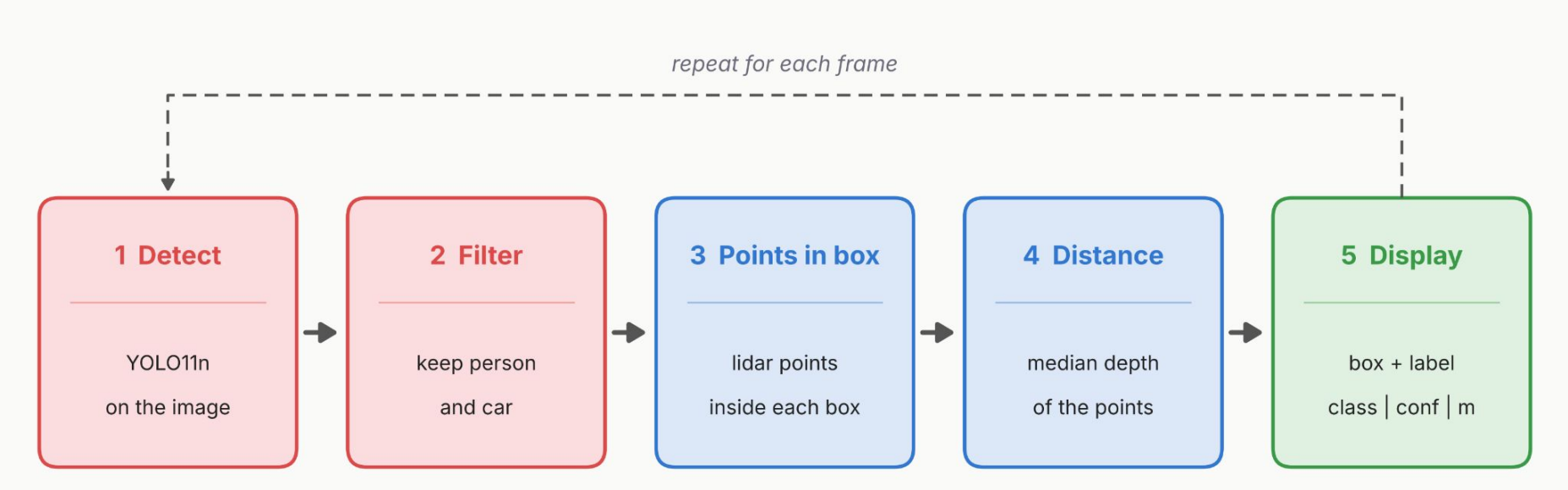
There are five sizes of YOLO11 models available for each task type. The model sizes are nano, small, medium, large, and extra-large.

When benchmarked on the COCO dataset for object detection, here is how YOLO11 performs.

Model	Size (px)	mAP 50-95 val
YOLO11n	640	39.5
YOLO11s	640	47.0
YOLO11m	640	51.5
YOLO11l	640	53.4
YOLO11x	640	54.7

Image from Ultralytics

Task b (ii):
Detection and segmented display



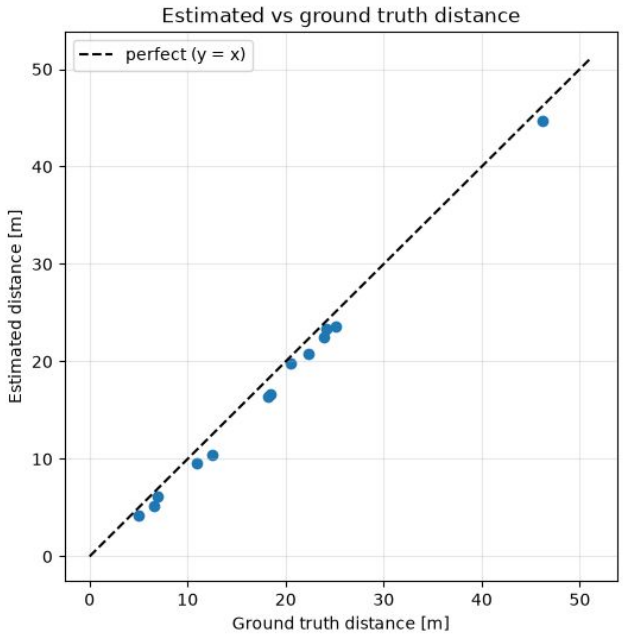
Task b (ii):
Detection and segmented display: result



Task b (iii):
Comparing detection to ground truth

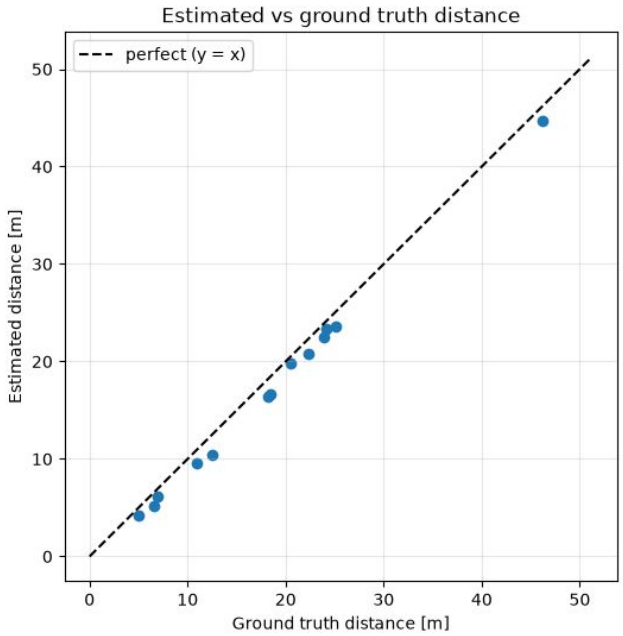
Metric	What it means
Error	Sign: + too far, − too close
MAE	Mean absolute error in metres
Bias	Systematic offset
Relative error	Error compared with the distance

Task b (iii):
Comparing detection to ground truth

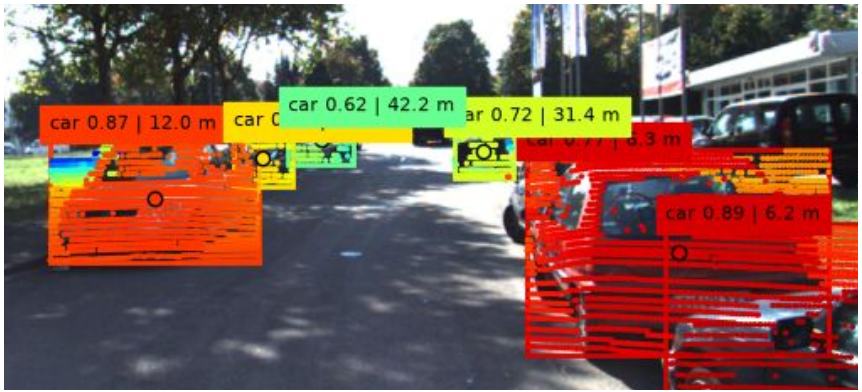


Metric	Value
MAE	1.33 m
Bias	−1.33 m
Relative error	9.6 %

Task b (iii):
Why did we see a MAE

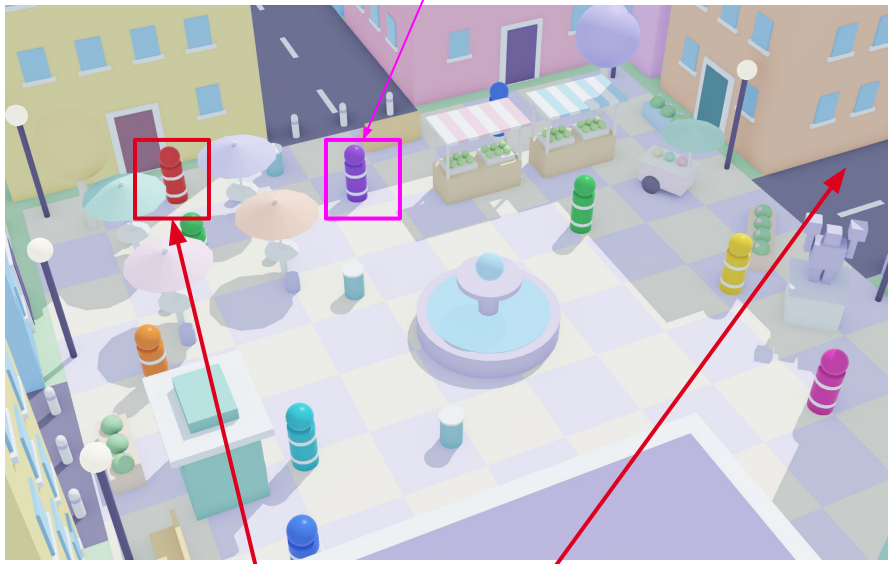


- Where on the car was the ground truth measured
- The points within the box but not on the car
  - Uses median of the center of bounding box points
- Lidar point density for objects far away

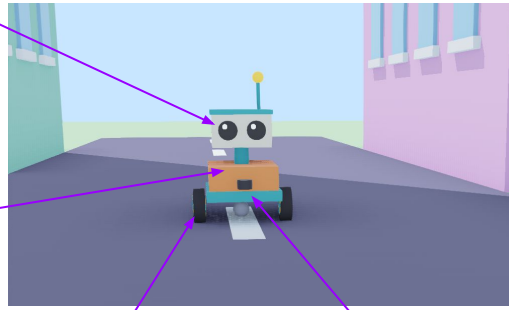


Q4 - Objective and overview

a) Wet floor (μ = 0.25): reach the red pillar, avoid obstacles, log every sensor, compute the wheel slip
b) Dry floor (μ = 1.0): same run, less slip
c) Plot the slip along the trajectory, wet vs dry



Our robot: Toby 🐶



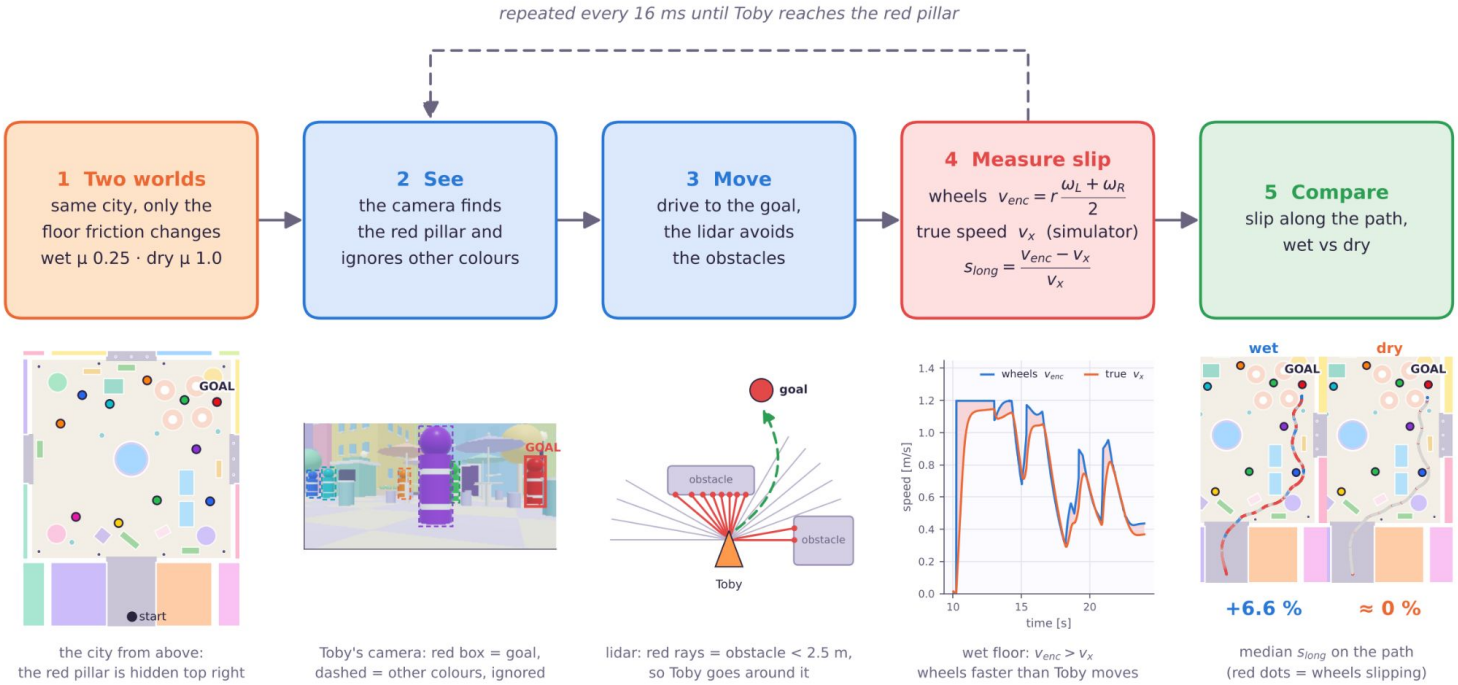
Camera:
- 256 x 128 - 69°
- Find the red pillar

- GPS
- MU
- Gyro
- Accelerometer

Wheel encoders:
- wheel rotation ωL, ωR
- v_enc = r(ωL+ωR)/2

Lidar:
- 181 rays over 180°
- 30 cm
- Distance to the obstacles

Q4 - How we solved it



the city from above:
the red pillar is hidden top right

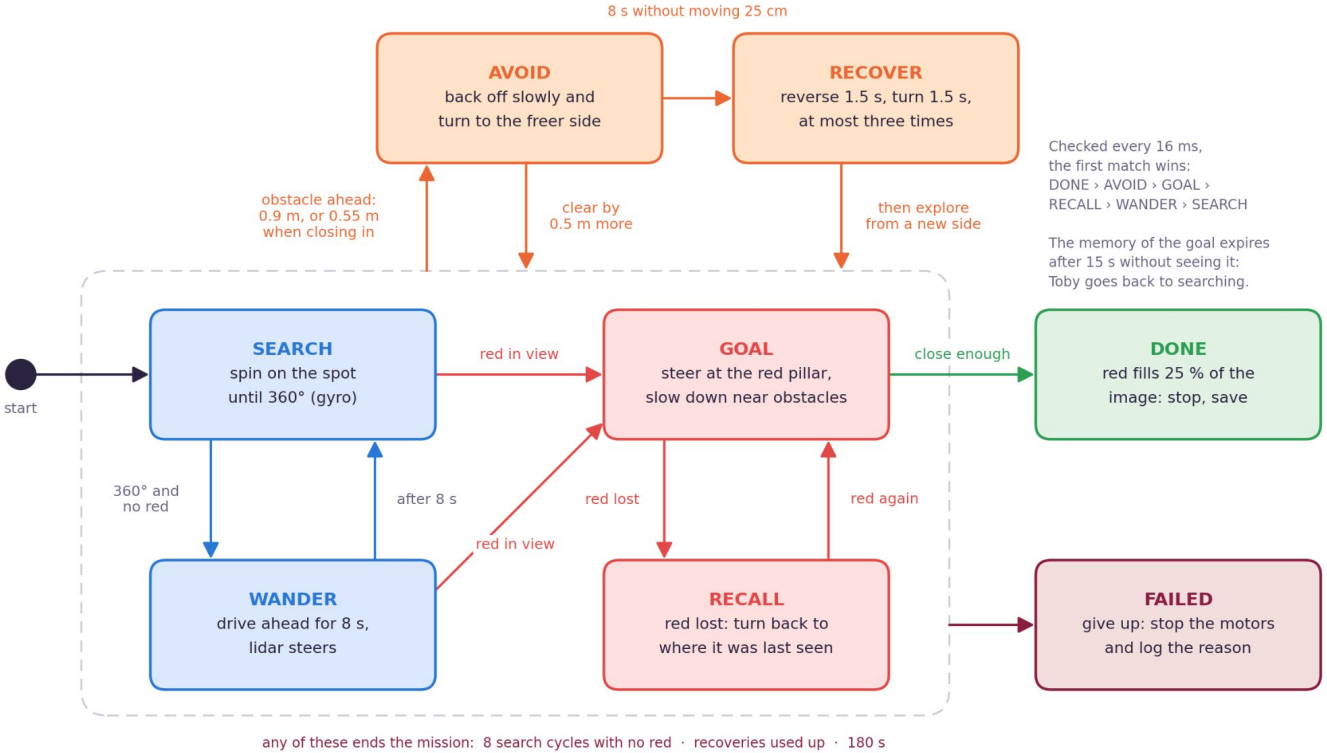
Toby's camera: red box = goal, dashed = other colours, ignored

lidar: red rays = obstacle < 2.5 m, so Toby goes around it

wet floor: $v_{enc} > v_x$
wheels faster than Toby moves

median s_{long} on the path
(red dots = wheels slipping)

Q4 - How Toby moves



Q4 - Key code

```python
# ---- wheel encoders -> v_enc
th_l, th_r = left_enc.getValue(), right_enc.getValue()
if prev_theta is None:
    prev_theta = (th_l, th_r)
om_l, om_r = (th_l - prev_theta[0]) / dt, (th_r - prev_theta[1]) / dt
prev_theta = (th_l, th_r)
v_enc = WHEEL_RADIUS * (om_l + om_r) / 2.0

# ---- ground truth -> v_x: world velocity projected on the robot's forward axis
vel = self_node.getVelocity()
R = self_node.getOrientation()
v_x = vel[0] * R[0] + vel[1] * R[3] + vel[2] * R[6]
pos = self_node.getPosition()
s_long = (v_enc - v_x) / v_x if abs(v_x) > V_X_MIN else float("nan")
```

```python
FRICTION = {
    #  run name     coulombFriction  forceDependentSlip  description
    "baseline": (0.25, 0.03, "wet: low friction, high slip"),
    "low_slip": (1.0, 0.0008, "dry: high friction, low slip"),
}
```

Q4 - Results

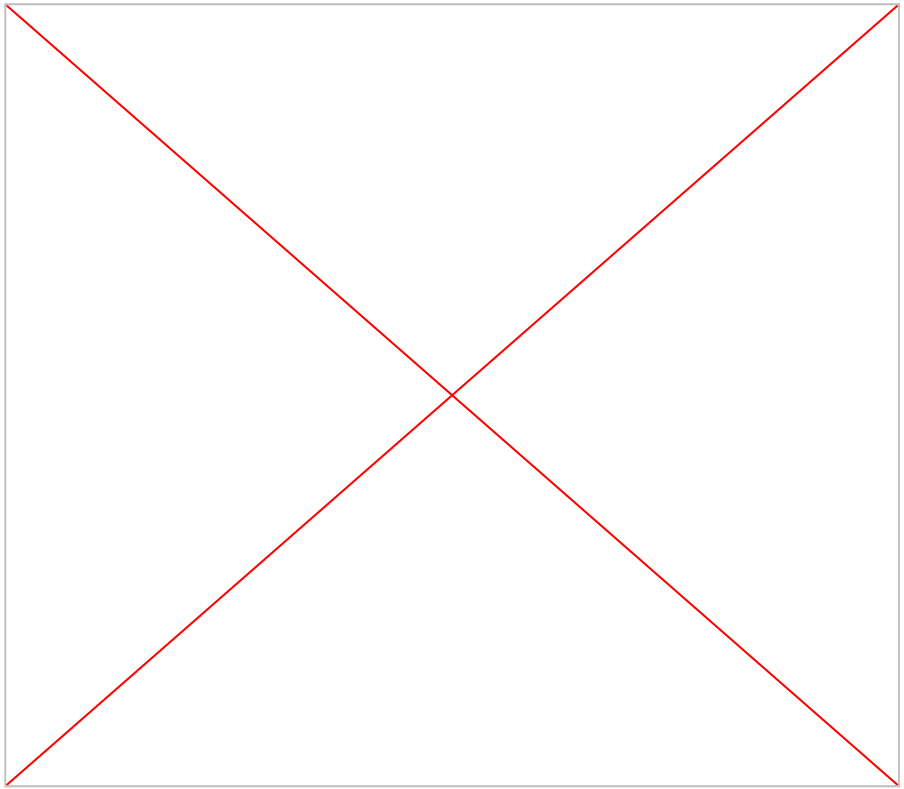
Baseline 🧊



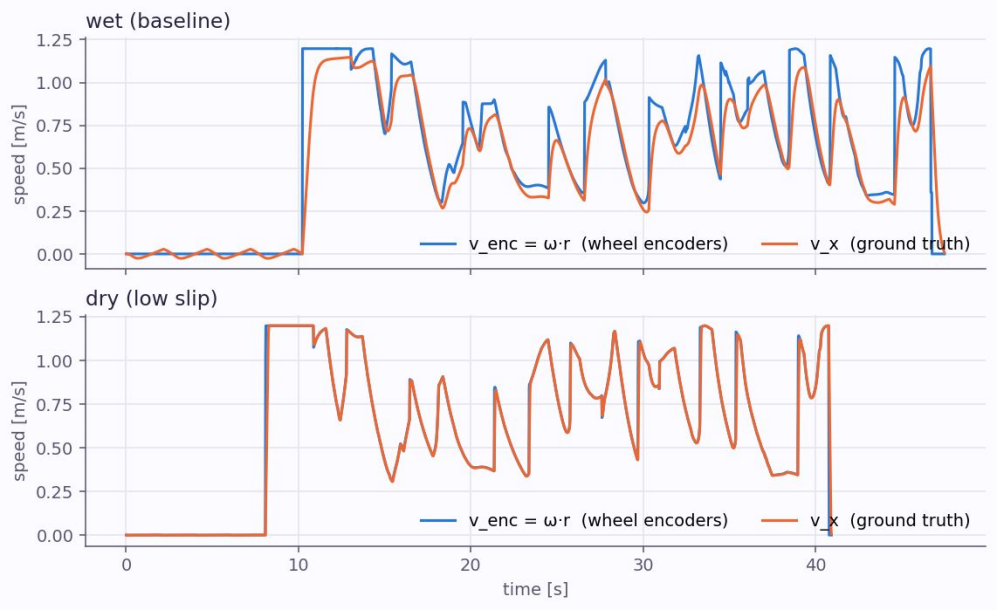
Low Slip 🔥



Q4 - Video Results



Q4 - Results: what slips looks like

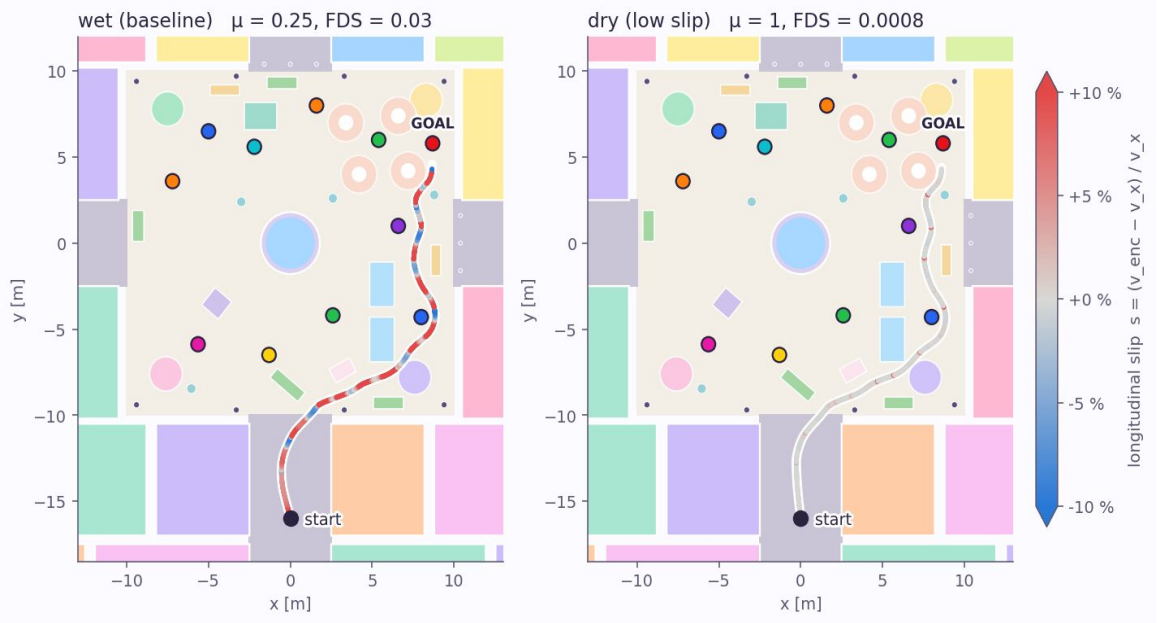


WET
The encoder line sits above the true speed for almost the whole run. That gap is the slip.

DRY
The two curves lie on top of each other. The wheels say what the body does.

THE FLAT STRETCHES
Zero forward speed: the search rotations on the spot. This is where the ratio is undefined and the samples are dropped.

Q4 - Results: Slip along the trajectory



THE COLOUR SCALE
- Red: the wheel turns faster than the robot.
- Blue: the wheel turns slower than the robot.
- Grey: no slip.

WET
Red along nearly the whole route, with blue patches where Toby brakes to steer around the market stalls.

DRY
Grey. The same mission, the same obstacles, and nothing to report.